Home About Queries

PIZZA HUT

Data Analysis

Using

SQL

PIZZA HUT

Home | About | Queries

DATASET

THE ANALYSIS WAS PERFORMED ON A RELATIONAL DATABASE FEATURING 48,000+ ROWS OF SALES DATA.

This project utilizes a comprehensive sales dataset from a pizza franchise, consisting of several interconnected tables: Orders, Order Details, Pizzas, and Pizza Types. The data encompasses thousands of transactions over a year long period, capturing critical details. By merging these relational tables, I performed deep dive analysis to uncover patterns in revenue, customer preferences, and operational efficiency.

PIZZA HUT

TABLES

01

ORDERS

Order ID
Order Date
Order Time

02

ORDER DETAILS

Order Details ID
Order ID
Pizza ID
Quantity

03

PIZZAS

Pizza ID
Pizza Type ID
Size
Price

04

PIZZA TYPES

Pizza Type ID
Name
Categoory
Ingredients

STATEMENT

Calculate the total revenue generated from pizza sales.

```
SELECT
    ROUND(SUM(order_details.quantity * pizzas.price),
            2) AS Total_Sales
FROM
    order_details
        JOIN
    pizzas ON pizzas.pizza_id = order_details.pizza_id
```

OUTPUT

Result Grid

	Total_Sales
▶	817860.05

STATEMENT

Identify the highest-priced pizza.

```sql
SELECT
    pizza_types.name, pizzas.price
FROM
    pizza_types
        JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
ORDER BY pizzas.price DESC
LIMIT 1
```

OUTPUT

Result Grid | Filter Rows:

name	price
The Greek Pizza	35.95

STATEMENT

Join the necessary tables to find the total quantity of each pizza category ordered

```sql
SELECT
    pizza_types.category,
    SUM(order_details.quantity) AS quantity
FROM
    pizza_types
        JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
        JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.category
ORDER BY quantity DESC
```

OUTPUT

Result Grid | Filter Rows:

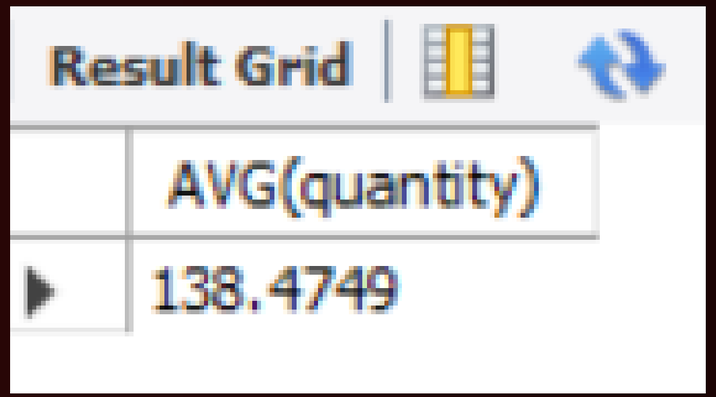
category	quantity
Classic	14888
Supreme	11987
Veggie	11649
Chicken	11050

STATEMENT

Group the orders by date and calculate the average number of pizzas ordered per day.

```
SELECT
    AVG(quantity)
FROM
    (SELECT
        orders.order_date, SUM(order_details.quantity) AS quantity
    FROM
        orders
    JOIN order_details ON orders.order_id = order_details.order_id
    GROUP BY orders.order_date) AS order_quantity;
```

OUTPUT



STATEMENT

Determine the top 3 most ordered pizza types based on revenue.

```
select pizza_types.name,
sum(order_details.quantity*pizzas.price) as revenue
from pizza_types join pizzas
on pizza_types.pizza_type_id=pizzas.pizza_type_id
join order_details
on order_details.pizza_id=pizzas.pizza_id
group by pizza_types.name
order by revenue
desc limit 3
```

OUTPUT

Result Grid | Filter Rows:

name	revenue
The Thai Chicken Pizza	43434.25
The Barbecue Chicken Pizza	42768
The California Chicken Pizza	41409.5

STATEMENT

Calculate the percentage contribution of each pizza type to total revenue.

```
select pizza_types.category,
round(sum(order_details.quantity*pizzas.price)/(select
round(sum(order_details.quantity * pizzas.price),2) as Total_Sales
from order_details join pizzas
on pizzas.pizza_id = order_details.pizza_id)*100,2) as revenue
from pizza_types join pizzas
on pizza_types.pizza_type_id=pizzas.pizza_type_id
join order_details
on order_details.pizza_id=pizzas.pizza_id
group by pizza_types.category
order by revenue desc;
```

OUTPUT

Result Grid | Filter Rows:

category	revenue
Classic	26.91
Supreme	25.46
Chicken	23.96
Veggie	23.68

STATEMENT

Analyze the cumulative revenue generated over time.

```
select order_date,
sum(revenue) over(order by order_date) as cum_revenue
from
(select orders.order_date,
sum(order_details.quantity*pizzas.price) as revenue
from order_details join pizzas
on order_details.pizza_id=pizzas.pizza_id
join orders
on orders.order_id=order_details.order_id
group by orders.order_date) as sales;
```

OUTPUT

Result Grid | Filter Rows:

order_date	cum_revenue	2
2015-01-01	2713.8500000000004	2
2015-01-02	5445.75	2
2015-01-03	8108.15	2
2015-01-04	9863.6	2
2015-01-05	11929.55	2
2015-01-06	14358.5	2
2015-01-07	16560.7	2
2015-01-08	19399.05	2
2015-01-09	21526.4	2
2015-01-10	23990.350000000002	2
2015-01-11	25862.65	2
2015-01-12	27781.7	2
2015-01-13	29831.300000000003	2
2015-01-14	32358.700000000004	2
2015-01-15	34343.50000000001	2
2015-01-16	36937.65000000001	2
2015-01-17	39001.75000000001	2
2015-01-18	40978.600000000006	2